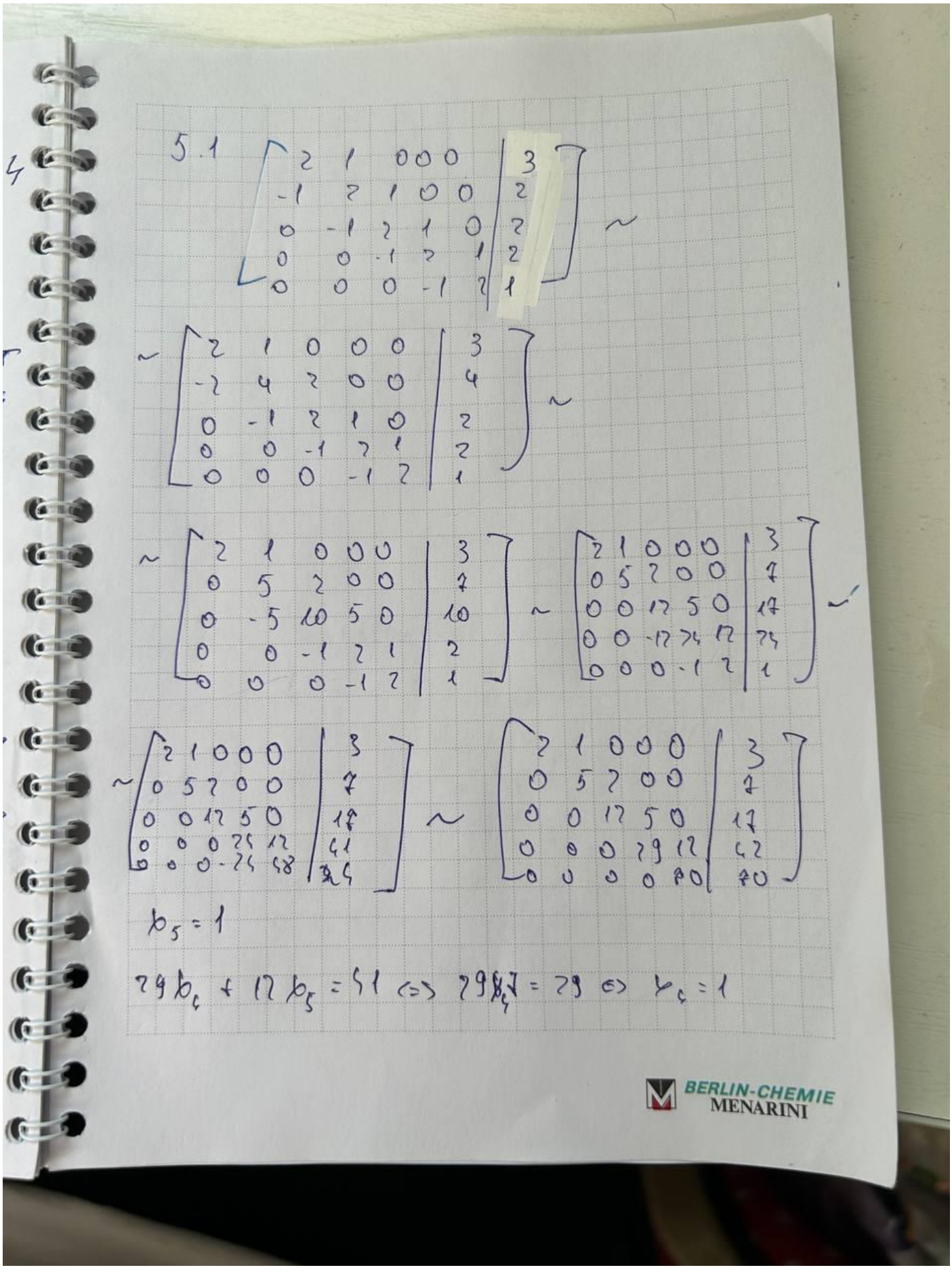
Problem 1 & 2



$6x_3 = -6 \Rightarrow x_3 = -1$

$-7x_2 - 4x_3 = 5$

$-7x_2 = 14$

$x_2 = -2$

$3x_1 - x_2 + 3x_3 = 11$

$3x_1 - 2 \cdot 3 = 11$

$3x_1 = 4$

$\begin{cases} x_1 = 4 \\ x_2 = -2 \\ x_3 = -1 \end{cases}$

•) LU function of A

$$\begin{bmatrix} 3 & -1 & 3 \\ 1 & 2 & -1 \\ 1 & 2 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix}$$

$$\begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix} = \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ l_{21}u_{11} & l_{21}u_{12}+u_{22} & l_{21}u_{13}+u_{23} \\ l_{31}u_{11} & l_{31}u_{12}+l_{32}u_{22} & l_{31}u_{13} \end{bmatrix}$$

$$\sim \left[\begin{array}{ccc|c} 3 & -1 & 3 & 11 \\ -3 & -6 & 6 & -8 \\ 1 & 2 & 0 & 0 \end{array}\right] \sim \left[\begin{array}{ccc|c} 3 & -1 & 3 & 11 \\ 0 & -7 & 9 & 5 \\ 1 & 2 & 0 & 0 \end{array}\right] \sim$$

$$\sim \left[\begin{array}{ccc|c} 3 & -1 & 3 & 11 \\ 0 & -7 & 9 & 5 \\ -3 & -6 & 0 & 0 \end{array}\right] \sim \left[\begin{array}{ccc|c} 3 & -1 & 3 & 11 \\ 0 & -7 & 9 & 5 \\ 0 & -7 & 3 & 11 \end{array}\right] \sim$$

$$\sim \left[\begin{array}{ccc|c} 3 & -1 & 3 & 11 \\ 0 & -7 & -9 & -5 \\ 0 & 0 & -6 & -6 \end{array}\right]$$

$u_{11} = 3$ $\quad l_{21} = \frac{1}{3}$

$u_{12} = -1$ $\quad l_{31} = \frac{1}{3}$

$u_{13} = 3$ $\quad l_{32} = 1$

$u_{22} = \frac{7}{3}$

$u_{23} = -3$

$u_{33} = 2$

$$A = \begin{bmatrix} 3 & -1 & 3 \\ 1 & 2 & -2 \\ 1 & 2 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{3} & 1 & 0 \\ \frac{1}{3} & 1 & 1 \end{bmatrix} =$$

$$= \begin{bmatrix} 3 & -1 & 3 \\ 0 & \frac{7}{3} & -3 \\ 0 & 0 & 2 \end{bmatrix}$$

Problem 3

Gauss-Seidel method is one of the common iterative methods to find solution of linear systems. Its working principle fairly simple, just same as normal algebraic way for finding values of unknown variables. However, in this method, when we express coefficients in a matrix form, diagonal dominant matrix is necessary to make sure our numerical
solution converges. Hence, below you will see function to check diagonal dominance of matrix and function for Gauss-Seidel method. Lets go!
Our matrix is:

$$A = \begin{pmatrix} 0 & 3 & 1 \\ -2 & 0 & -3 \\ 1 & 4 & 0 \end{pmatrix}$$

Here is my code for check if iteration method converge!

```
# Check if the matrix is diagonal dominant
def check_diaDom(A):
r = len(A)
# Always good idea to see if the code is working the way it should be # so, I have
included print option to check throughout the process
diagnl = 0 non_diagnl = 0
for i in range(r): for j in range(r):
# print(f'Loop i = {i}, j = {j}\n') # print(A[i, j])
if i == j:
diagnl += abs(A[i][j])
elif i != j: # here you can simply use else rather than else if
non_diagnl += abs(A[i][j]) if diagnl >= non_diagnl: verdict = True
# print('Diagonal dominant Matrix')
else:
# print('Non-diagonal dominant Matrix')
verdict = False return verdict
        def GS_method(A, Y, X):
n = len(A) # to find no of rows or col of sq matrix A
for j in range(0, n):
# old_val = X[i]
summ_val = Y[j] for i in range(0, n):
if j != i:
summ_val -= A[j][i] * X[i]
# print(summ_val)
X[j] = summ_val / A[j][j]
# print(X[i])
return X
        A = [[0, 3, 1], [-2, 0, -3], [1, 4, 0]] # coefficient of unknown variables Y
= [14, 5, -8] # terms that appears after '=' sign in the equation X = [-2 / 3, 5, 4]
# initial guess for our method
if check_diaDom(A) == 1:
for i in range(10):
X = GS_method(A, Y, X)
print(X)
else:
print("Non-Diagonal matrix")
```

Output is

```
[Running] python -u "c:\Use
Non-Diagonal matrix
```

So it means that it never converge at any initial guess! Because the main condition to have diagonal matrix.

Problem 4

The direct method which will be used - LU decomposition. I choose LU decomposition as the direct method because it's
efficient for solving large systems of equations, can be used for any type of coefficient matrices and doesn't require matrix inversion.
Solution of LU decompasition is

```
[ 87.49970902 -55.75380417 17.77411635 32.04744366 11.74397126
  -94.36514182   9.18489557 -8.10628054 99.97509067 52.53605364J
```

Here is code:

```
import numpy as np
from scipy.linalg import lu_factor, lu_solve
```

```
import numpy as np
from scipy.linalg import lu_factor, lu_solve
        A = np.array( [
[0, 1, 5, -7, 23, -1, 7, 8, 1, -5],
[17, 0, -24, -75, 100, -18, 10, -8, 9, -50],
[3, -2, 15, 0, -78, -90, -70, 18, -75, 1],
[5, 5, -10, 0, -72, -1, 80, -3, 10, -18],
[100, -4, -75, -8, 0, 83, -10, -75, 3, -8],
[70, 85, -4, -9, 2, 0, 3, -17, -1, -21],
[1, 15, 100, -4, -23, 13, 0, 7, -3, 17],
[16, 2, -7, 89, -17, 11, -73, 0, -8, -23],
[51, 47, -3, 5, -10, 18, -99, -18, 0, 12],
[1, 1, 1, 1, 1, 1, 1, 1, 1, 0],
        ]
)
b = np.array([10, -40, -17, 43, -53, 12, -60, 100, 0, 100])
LU = lu_factor(A)
x_lu = lu_solve(LU, b)
print(x_lu)
```

For direct I will use previous code to check if it’s converge.

Code again

```
# Check if the matrix is diagonal dominant
def check_diaDom(A):
r = len(A)
# Always good idea to see if the code is working the way it should be # so, I have
included print option to check throughout the process
diagnl = 0 non_diagnl = 0
for i in range(r): for j in range(r):
# print(f'Loop i = {i}, j = {j}\n') # print(A[i, j])
if i == j:
diagnl += abs(A[i][j])
elif i != j: # here you can simply use else rather than else if
non_diagnl += abs(A[i][j])
# print('Sum of diagonal elements\n', diagnl) # print('Sum of non-diagonal
elements\n', non_diagnl)
if diagnl >= non_diagnl: verdict = True
# print('Diagonal dominant Matrix')
else:
# print('Non-diagonal dominant Matrix')
verdict = False
return verdict
def GS_method(A, Y, X):
n = len(A) # to find no of rows or col of sq matrix A
for j in range(0, n):
# old_val = X[i]
summ_val = Y[j] for i in range(0, n):
if j != i:
summ_val -= A[j][i] * X[i]
# print(summ_val)
X[j] = summ_val / A[j][j]
# print(X[i])
return X
A = [
[0, 1, 5, -7, 23, -1, 7, 8, 1, -5],
[17, 0, -24, -75, 100, -18, 10, -8, 9, -50],
[3, -2, 15, 0, -78, -90, -70, 18, -75, 1],
[5, 5, -10, 0, -72, -1, 80, -3, 10, -18],
[100, -4, -75, -8, 0, 83, -10, -75, 3, -8],
[70, 85, -4, -9, 2, 0, 3, -17, -1, -21],
[1, 15, 100, -4, -23, 13, 0, 7, -3, 17],
```

```
[16, 2, -7, 89, -17, 11, -73, 0, -8, -23],
[51, 47, -3, 5, -10, 18, -99, -18, 0, 12],
[1, 1, 1, 1, 1, 1, 1, 1, 1, 0],
] # coefficient of unknown variables
if check_diaDom(A) == 1:
for i in range(10):
X = GS_method(A, Y, X)
print(X)
else:
print("Non-Diagonal matrix")
```

Output is

```
Non-Diagonal matrix
```

None of the iterative methods won't converge to the answer. The matrix is singular or ill-conditioned and not diagonally dominant.As a result we can conclude, that these iteratives methods don't converge.

Problem 5

GAUSS-SEIDEL ITERATION METHOD

$$x_1^{(k+1)} = \frac{1}{9}\left[b_1 - x_2^{(k)} - x_3^{(k)}\right]$$

$$x_2^{(k+1)} = \frac{1}{10}\left[b_2 - 2x_1^{(k+1)} - 3x_3^{(k)}\right]$$

$$x_3^{(k+1)} = \frac{1}{11}\left[b_3 - 3x_1^{(k+1)} - 4x_2^{(k+1)}\right]$$

Here it is algorithm from Reading where coefficients in front of brackets arecoefficient in front of x1 x2 x3

Formula for error is $\dfrac{\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|_\infty}{\|\mathbf{x}^{(k)}\|_\infty}$

Here is python code which implements this algorithm

```
# Importing modules
from numpy import *
# Settingup initial guess
xg = 1 yg = 1 zg = 1
# Error for entering in the loop
error = 1
# Iteration counter
count = 0
while error > 1.0e-6:
count += 1 x = (20 - 3 * yg - 2 * zg) / 8
# Here x is x_new (obtained in this step only)
y = (40.02 - 16 * x - 4.001 * zg) / 6
# Here x and y are x_new and y_new (obtained in this step only)
z = 10.01 - 4 * x - 1.501 * y
# Evaluating and Plotting the errors
error = abs(x - xg) + abs(y - yg) + abs(z - zg)
# Setting the guess for the next iteration
xg = x yg = y zg = z
print(
f"Iteration {count}: x={round(x, 5)}, y={round(y, 5)}, z={round(z, 5)},
error={error}"
)
print(f"x={round(x,5)}, y={round(y,5)}, z={round(z,5)}")
```

And it is result is amazing I stooped my code when iteration are 188622

```
Iteration 188585: x=3.799121411430874c>44, y=8.389432958266394e+44, z=-2.7789024516081352e+45, error=2.2058005485769757e+42
Iteration 188586: x=3.80121876967944e+44, y=8.394064462352408e>44, z=-2.7804365836672724e+45, error=2.2070182916951125e+42
Iteration 188587: x=3.803317285786028e+44, y=8.398698523325189e+44, z=-2.781971562665522e+45, error=2.2082367070865505e+42
Iteration 188588: x=3.8054169604168595e+44, y=8.403335142596298e+44, z=-2.783507389070448e+45, error=2.209455795120097e+42
Iteration 188589: x=3.807517794202508e+44, y=8.407974321578084e+44, z=-2.7850440633498736e+45, error=2.2106755561689955e+42
Iteration 188590: x=3.8096197877829025e+44, y=8.412616061683668e+44, z=-2.7865815859718794e+45, error=2.2118959906035588e+42
Iteration 188591: x=3.811722941798323e+44, y=8.417260364326956e+44, z=-2.7881199574048052e+45, error=2.2131170987967137e+42
Iteration 188592: x=3.813827256889404e+44, y=8.421907230922634e+44, z=-2.789659178117249e+45, error=2.214338881119565e+42
Iteration 188593: x=3.815932733697135e+44, y=8.426556662886162e+44, z=-2.7911992485780665e+45, error=2.2155613379435343e+42
Iteration 188594: x=3.818039372862855e+44, y=8.431208661633794e+44, z=-2.7927401692563744e+45, error=2.216784469643133e+42
Iteration 188595: x=3.820147175028263e+44, y=8.435863228582555e+44, z=-2.7942819406215468e+45, error=2.2180082765893076e+42
Iteration 188596: x=3.822256140835409e+44, y=8.440520365150026e+44, z=-2.7958245631432176e+45, error=2.219232759155777e+42
Iteration 188597: x=3.824366270926697e+44, y=8.4451800727555e+44, z=-2.797368037291279e+45, error=2.2204579177142802e+42
Iteration 188598: x=3.8264775659448855e+44, y=8.449842352817654e+44, z=-2.7989123635358838e+45, error=2.2216837526390906e+42
Iteration 188599: x=3.8285900265330895e+44, y=8.454507206756881e+44, z=-2.8004575423474438e+45, error=2.2229102643030562e+42
Iteration 188600: x=3.830703653334779e+44, y=8.459174635994129e+44, z=-2.80200357419663e+45, error=2.224137453079817e+42
Iteration 188601: x=3.8328184469937764e+44, y=8.46384464195H25e+44, z=-2.8035504595543745e+45, error=2.2253653193439634e+42
Iteration 188602: x=3.834934408154264e+44, y=8.468517226050385e+44, z=-2.8050981988918682e+45, error=2.2265938634685813e+42
Iteration 188603: x=3.837051537460776e+44, y=8.473192389715205e+44, z=-2.8066467926805625e+45, error=2.227823085827469e+42
Iteration 188604: x=3.839169835558204c+44, y=8.477870134369673e+44, z=-2.8081962413921697e+45, error=2.2290529867968016e+42
Iteration 188605: x=3.841289303091797e+44, y=8.482550461438663e+44, z=-2.80974654549866 2e+45, error=2.230283566750378e+42
Iteration 188606: x=3.843409940707156e+44, y=8.487233372347829e+44, z=-2.8112977054722715e+45, error=2.2315148260620754e+42
Iteration 188607: x=3.845531749050243e+44, y=8.491918868523616e+44, z=-2.812849721785492e+45, error=2.2327467651080695e+42
Iteration 188608: x=3.847654728767374e+44, y=8.49660695139326e+44, z=-2.814402594911078e+45, error=2.233979384263347e+42
Iteration 188609: x=3.849778880505222e+44, y=8.501297622384781e+44, z=-2.8159563253220445e+45, error=2.2352126839034495e+42
Iteration 188610: x=3.8519042049108185e+44, y=8.505990882926985e+44, z=-2.817510913491668e+45, error=2.236446664403443e+42
Iteration 188611: x=3.85403070263155e+44, y=8.510686734449474e+44, z=-2.819066359893486e+45, error=2.237681326140137e+42
Iteration 188612: x=3.8561583743151625e+44, y=8.51538517838263e+44, z=-2.8206226650012978e+45, error=2.2389166694886762e+42
Iteration 188613: x=3.858287220609758e+44, y=8.520086216157634e+44, z=-2.822179829289164e+45, error=2.240152694826187e+42
Iteration 188614: x=3.860417242163797e+44, y=8.524789849206451e+44, z=-2.8237378532314073e+45, error=2.241389402528845e+42
Iteration 188615: x=3.862548439626099c+44, y=8.529496078961838e+44, z=-2.8252967373026114e+45, error=2.242626792973063e+42
Iteration 188616: x=3.864680813645839e+44, y=8.534204906857344e+44, z=-2.826856481977623e+45, error=2.243864866536125e+42
Iteration 188617: x=3.8668143648725535e+44, y=8.538916334327388e+44, z=-2.82841708773155e+45, error=2.245103623594841e+42
Iteration 188618: x=3.868949093956135e+44, y=8.54363036280686e+44, z=-2.8299785550397637e+45, error=2.2463430645271287e+42
Iteration 188619: x=3.8710850015468366e+44, y=8.548346993731928e+44, z=-2.8315408843778968e+45, error=2.2475831897100352e+42
Iteration 188620: x=3.873222088295269e+44, y=8.553066228539225e+44, z=-2.8331040762218452e+45, error=2.2488239995213996e+42
Iteration 188621: x=3.875360354852404e+44, y=8.557788068666261e+44, z=-2.8346681310477674e+45, error=2.2500654943392194e+42
Iteration 188622?: x=3.87749980186957e+44, y=8.562512515551343e+44, z=-2.8362330493320847e+45, error=2.251307674542126e+42
```

Here is error for MEM. In this way, we get a huge amount of iterations, they don't end up. Therefore, equations are divergent and probably doesn't have unique solution

```
[Done] exited with code=0 in 0.434 seconds

[Running] python -u "c:\Users\Duma\OneDrive\Рабочий стол\065 businessbox-ma
Iteration 1: x=1.875, y=1.00317, z=1.00425, error=0.8824134999999995
Iteration 2: x=1.87275, y=1.00633, z=1.00849, error=0.009659896277250057
Iteration 3: x=1.8705, y=1.0095, z=1.01273, error=0.009656293263551996
Iteration 4: x=1.86825, y=1.01266, z=1.01697, error=0.009652690958482202
Iteration 5: x=1.86601, y=1.01583, z=1.02121, error=0.009649089361618568
Iteration 6: x=1.86376, y=1.01899, z=1.02545, error=0.009645488472537433
Iteration 7: x=1.86152, y=1.02215, z=1.02968, error=0.009641888290815581
Iteration 8: x=1.85927, y=1.02531, z=1.03392, error=0.00963828881602935
Iteration 9: x=1.85703, y=1.02847, z=1.03815, error=0.009634690047754635
Iteration 10: x=1.85479, y=1.03163, z=1.04237, error=0.009631091985572215
Iteration 11: x=1.85254, y=1.03479, z=1.0466, error=0.009627494629054878
Iteration 12: x=1.8503, y=1.03795, z=1.05082, error=0.009623897977780071
Iteration 13: x=1.84806, y=1.04111, z=1.05504, error=0.009620302031326355
Iteration 14: x=1.84582, y=1.04427, z=1.05926, error=0.009616706789270513
Iteration 15: x=1.84358, y=1.04743, z=1.06348, error=0.009613112251188216
Iteration 16: x=1.84134, y=1.05058, z=1.0677, error=0.00960951841665491 6
Iteration 17: x=1.83911, y=1.05374, z=1.07191, error=0.009605925285250727
Iteration 18: x=1.83687, y=1.05689, z=1.07612, error=0.009602332856551321
Iteration 19: x=1.83464, y=1.06005, z=1.08033, error=0.009598741130133925
Iteration 20: x=1.8324, y=1.0632, z=1.08454, error=0.009595150105573325
Iteration 21: x=1.83017, y=1.06635, z=1.08874, error=0.009591559782451187
Iteration 22: x=1.82793, y=1.0695, z=1.09294, error=0.009587970160339188
Iteration 23: x=1.8257, y=1.07265, z=1.09715, error=0.00958438123881633
Iteration 24: x=1.82347, y=1.07581, z=1.10134, error=0.009580793017461842
Iteration 25: x=1.82124, y=1.07896, z=1.10554, error=0.009577205495849617
Iteration 26: x=1.81901, y=1.0821, z=1.10973, error=0.009573618673559103
Iteration 27: x=1.81678, y=1.08525, z=1.11393, error=0.009570032550165308
Iteration 28: x=1.81455, y=1.0884, z=1.11812, error=0.009566447125246347
Iteration 29: x=1.81232, y=1.09155, z=1.1223, error=0.009562862393877767
Iteration 30: x=1.81009, y=1.09469, z=1.12649, error=0.009559278369139939
Iteration 31: x=1.80787, y=1.09784, z=1.13067, error=0.009555695037105849
Iteration 32: x=1.80564, y=1.10098, z=1.13486, error=0.009552112401856494
Iteration 33: x=1.80342, y=1.10413, z=1.13904, error=0.009548530462965887
Iteration 34: x=1.80119, y=1.10727, z=1.14321, error=0.009544949220013477
Iteration 35: x=1.79897, y=1.11041, z=1.14739, error=0.009541368672572936
Iteration 36: x=1.79675, y=1.11356, z=1.15156, error=0.009537788820224824
Iteration 37: x=1.79453, y=1.1167, z=1.15573, error=0.009534209662545257
Iteration 38: x=1.79231, y=1.11984, z=1.1599, error=0.009530631199111683
Iteration 39: x=1.79009, y=1.12298, z=1.16407, error=0.009527053429501109
Iteration 40: x=1.78787, y=1.12612, z=1.16823, error=0.009523476353288984
Iteration 41: x=1.78565, y=1.12926, z=1.1724, error=0.009519899970054535
Iteration 42: x=1.78343, y=1.13239, z=1.17656, error=0.009516324279375 21
Iteration 43: x=1.78121, y=1.13553, z=1.18072, error=0.009512749280825128
Iteration 44: x=1.779, y=1.13867, z=1.18487, error=0.009509174973984624
Iteration 45: x=1.77678, y=1.1418, z=1.18903, error=0.009505601358430482
Iteration 46: x=1.77457, y=1.14494, z=1.19318, error=0.009502028433373904
Iteration 47: x=1.77235, y=1.14807, z=1.19733, error=0.009498456199487526
Iteration 48: x=1.77014, y=1.15121, z=1.20148, error=0.009494884655252944
```

```
Iteration 107: x=1.64095, y=1.33489, z=1.44251, error=0.009285369665514533
Iteration 108: x=1.63879, y=1.33798, z=1.44654, error=0.009281838749338826
Iteration 109: x=1.63662, y=1.34107, z=1.45056, error=0.009278308499730598
Iteration 110: x=1.63446, y=1.34417, z=1.45457, error=0.009274778908644477
Iteration 111: x=1.63229, y=1.34726, z=1.45859, error=0.009271249983582308
Iteration 113: x=1.63013, y=1.35035, z=1.4636, error=0.009267723173154687
Iteration 113: x=1.62797, y=1.35344, z=1.46662, error=0.009264194121569105
Iteration 114: x=1.62581, y=1.35653, z=1.47063, error=0.009260667118457208
Iteration 115: x=1.62365, y=1.35961, z=1.47464, error=0.009257140908884441
Iteration 116: x=1.62149, y=1.3627, z=1.47864, error=0.009253615294485096
Iteration 117: x=1.61933, y=1.36579, z=1.48264, error=0.009250090340950923
Iteration 118: x=1.61717, y=1.36887, z=1.48665, error=0.009246566047861116
Iteration 119: x=1.61501, y=1.37196, z=1.49065, error=0.009243042414391791
Iteration 120: x=1.61285, y=1.37504, z=1.49464, error=0.009239519441121953
Iteration 121: x=1.6107, y=1.37813, z=1.49864, error=0.009235997127028384
Iteration 122: x=1.60854, y=1.38121, z=1.50263, error=0.009232475471489199
Iteration 123: x=1.60639, y=1.3843, z=1.50662, error=0.009228954474281181
Iteration 124: x=1.60423, y=1.38738, z=1.51061, error=0.009225434134583779
Iteration 125: x=1.60208, y=1.39046, z=1.5146, error=0.009221914451311333
Iteration 126: x=1.59993, y=1.39354, z=1.51859, error=0.009218395428428394
Iteration 127: x=1.59778, y=1.39662, z=1.52257, error=0.009214877060125533
Iteration 128: x=1.59563, y=1.3997, z=1.52655, error=0.009211359348445303
Iteration 129: x=1.59348, y=1.40278, z=1.53053, error=0.009207842292363155
Iteration 130: x=1.59133, y=1.40586, z=1.53451, error=0.009204325891656654
Iteration 131: x=1.58918, y=1.40893, z=1.53848, error=0.009200810145904459
Iteration 132: x=1.58703, y=1.41201, z=1.54245, error=0.009197295054668414
Iteration 133: x=1.58488, y=1.41509, z=1.54642, error=0.009193780617574143
Iteration 134: x=1.58274, y=1.41816, z=1.55039, error=0.009190266834351829
Iteration 135: x=1.58059, y=1.42124, z=1.55436, error=0.009186753703994822
Iteration 136: x=1.57845, y=1.42431, z=1.55832, error=0.009183241226679462
Iteration 137: x=1.5763, y=1.42739, z=1.56228, error=0.009179729401785908
Iteration 138: x=1.57416, y=1.43046, z=1.56624, error=0.009176218228891031
Iteration 139: x=1.57202, y=1.43353, z=1.5702, error=0.009172707707572725
Iteration 140: x=1.56988, y=1.4366, z=1.57416, error=0.009169197837427772
Iteration 141: x=1.56773, y=1.43967, z=1.57811, error=0.009165688617576864
Iteration 142: x=1.56559, y=1.44274, z=1.58206, error=0.009162180048855394
Iteration 143: x=1.56345, y=1.44581, z=1.58601, error=0.009158672129619516
Iteration 144: x=1.56132, y=1.44888, z=1.58996, error=0.009155164859051128
Iteration 145: x=1.55918, y=1.45195, z=1.59391, error=0.009151658239124671
Iteration 146: x=1.55704, y=1.45502, z=1.59785, error=0.009148152267021814
Iteration 147: x=1.55491, y=1.45809, z=1.60179, error=0.009144646943111623
Iteration 148: x=1.55277, y=1.46115, z=1.60573, error=0.009141142266587146
Iteration 149: x=1.55064, y=1.46422, z=1.60967, error=0.009137638236214008
Iteration 150: x=1.5485, y=1.46728, z=1.6136, error=0.009134134856372045
Iteration 151: x=1.54637, y=1.47035, z=1.61754, error=0.009130632121041371
Iteration 152: x=1.54424, y=1.47341, z=1.62147, error=0.009127130031798547
Iteration 153: x=1.5421, y=1.47647, z=1.6254, error=0.009123628588222354
Iteration 154: x=1.53997, y=1.47954, z=1.62932, error=0.009120127788988891
Iteration 155: x=1.53784, y=1.4826, z=1.63325, error=0.009116627636338327
Iteration 156: x=1.53571, y=1.48566, z=1.63717, error=0.009113128127266945
Iteration 157: x=1.53358, y=1.48872, z=1.64109, error=0.009109629262132213
```

Problem 5

After Gauss-Jordan I've got this result:

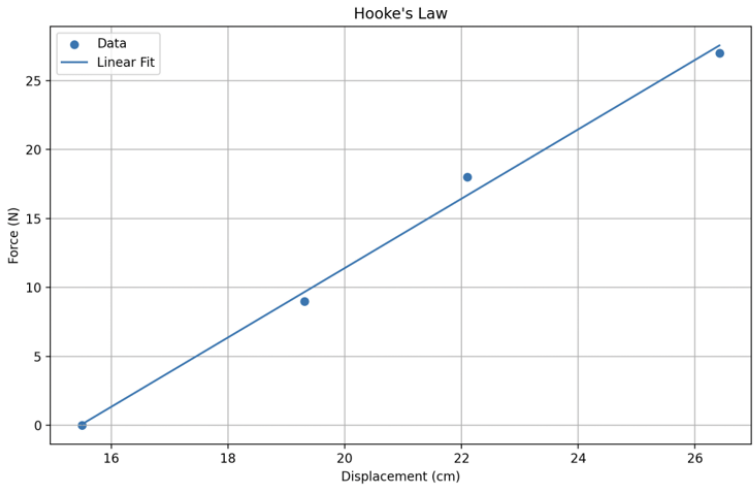
```
Reduced Row Echelon Form:
 [[ 1.  0.  0.  6.]
 [-0.  1.  0. -1.]
 [-0. -0.  1. -2.]]
```

But we can't implement the Gauss-Seidel, because the equations are divergent and doesn't have unique solution, so this matrix is not diagonal

```
5330685 -10725.07 28625.9468 -28.6281 2.86e+04 1.0000002831138284
5330686 -10725.073 28625.9549 -28.6281 2.86e+04 1.0000002831137955
5330687 -10725.0761 28625.963 -28.6281 2.86e+04 1.0000002831137624
5330688 -10725.0791 28625.9711 -28.6281 2.86e+04 1.0000002831137296
5330689 -10725.0821 28625.9792 -28.6281 2.86e+04 1.0000002831136965
5330690 -10725.0852 28625.9873 -28.6281 2.86e+04 1.0000002831136636
5330691 -10725.0882 28625.9954 -28.6281 2.86e+04 1.0000002831136308
5330692 -10725.0913 28626.0035 -28.6282 2.86e+04 1.0000002831135977
5330693 -10725.0943 28626.0116 -28.6282 2.86e+04 1.0000002831135646
5330694 -10725.0973 28626.0198 -28.6282 2.86e+04 1.0000002831135315
5330695 -10725.1004 28626.0279 -28.6282 2.86e+04 1.0000002831134986
5330696 -10725.1034 28626.036 -28.6282 2.86e+04 1.000000283113466
5330697 -10725.1064 28626.0441 -28.6282 2.86e+04 1.000000283113433
5330698 -10725.1095 28626.0522 -28.6282 2.86e+04 1.0000002831133998
5330699 -10725.1125 28626.0603 -28.6282 2.86e+04 1.000000283113367
5330700 -10725.1156 28626.0684 -28.6282 2.86e+04 1.0000002831133339
5330701 -10725.1186 28626.0765 -28.6282 2.86e+04 1.000000283113301
5330702 -10725.1216 28626.0846 -28.6282 2.86e+04 1.0000002831132682
5330703 -10725.1247 28626.0927 -28.6282 2.86e+04 1.0000002831132353
5330704 -10725.1277 28626.1008 -28.6282 2.86e+04 1.0000002831132022
5330705 -10725.1307 28626.1089 -28.6283 2.86e+04 1.000000283113169
5330706 -10725.1338 28626.117 -28.6283 2.86e+04 1.000000283113365
5330707 -10725.1368 28626.1251 -28.6283 2.86e+04 1.0000002831131034
5330708 -10725.1398 28626.1332 -28.6283 2.86e+04 1.0000002831130705
5330709 -10725.1429 28626.1413 -28.6283 2.86e+04 1.0000002831130375
5330710 -10725.1459 28626.1494 -28.6283 2.86e+04 1.0000002831130046
5330711 -10725.149 28626.1575 -28.6283 2.86e+04 1.0000002831129715
5330712 -10725.152 28626.1656 -28.6283 2.86e+04 1.0000002831129386
5330713 -10725.155 28626.1737 -28.6283 2.86e+04 1.0000002831129056
5330714 -10725.1581 28626.1818 -28.6283 2.86e+04 1.0000002831128727
5330715 -10725.1611 28626.19 -28.6283 2.86e+04 1.0000002831128398
5330716 -10725.1641 28626.1981 -28.6283 2.86e+04 1.0000002831128065
5330717 -10725.1672 28626.2062 -28.6284 2.86e+04 1.0000002831127737
5330718 -10725.1702 28626.2143 -28.6284 2.86e+04 1.0000002831127408
5330719 -10725.1733 28626.2224 -28.6284 2.86e+04 1.000000283112708
5330720 -10725.1763 28626.2305 -28.6284 2.86e+04 1.0000002831126749
5330721 -10725.1793 28626.2386 -28.6284 2.86e+04 1.000000283112642
^Z5330722 -10725.1824 28626.2467 -28.6284 2.86e+04 1.0000002831126609
5330723 -10725.1854 28626.2548 -28.6284 2.86e+04 1.0000002831125763
```

```
import numpy as np
def gauss_jordan_elimination(A): n = len(A)
for i in range(n):
max_index = i
for j in range(i+1, n):
if abs(A[j, i]) > abs(A[max_index, i]):
max_index = j A[[i, max_index]] = A[[max_index, i]] pivot = A[i, i] if pivot != 0:
A[i] = A[i] / pivot
for j in range(n):
if j != i: factor = A[j, i] A[j] -= factor * A[i]
return A A = np.array([[1, 1, -1, 7], [1, -1, 2, 3],
[2, 1, 1, 9]], dtype=float)
reduced_row_echelon_form = gauss_jordan_elimination(A)
print("Reduced Row Echelon Form:\n", reduced_row_echelon_form)
# Gauss-Seidel iteration
results = [6, -1, 2] x_guess = 1 y_guess = 1 z_guess = 1 error = 1 count = 0 while
error > 1e-6: x = (20 - 3 * y_guess - 2 * z_guess) / 8 y = (40.02 - 16 * x - 4.001 *
z_guess) / 6 z = (10.01 - 4 * x - 1.501 * y) / 2
new_error = max(abs(results[0] - x_guess), abs(results[1] - y_guess), abs(results[2]
- z_guess))
x_guess = x
y_guess = y z_guess = z count += 1
print(count, round(x_guess, 4), round(y_guess, 4), round(z_guess, 4),
"{:.2e}".format(error), new_error / error)
error = new_error
```

Problem 6

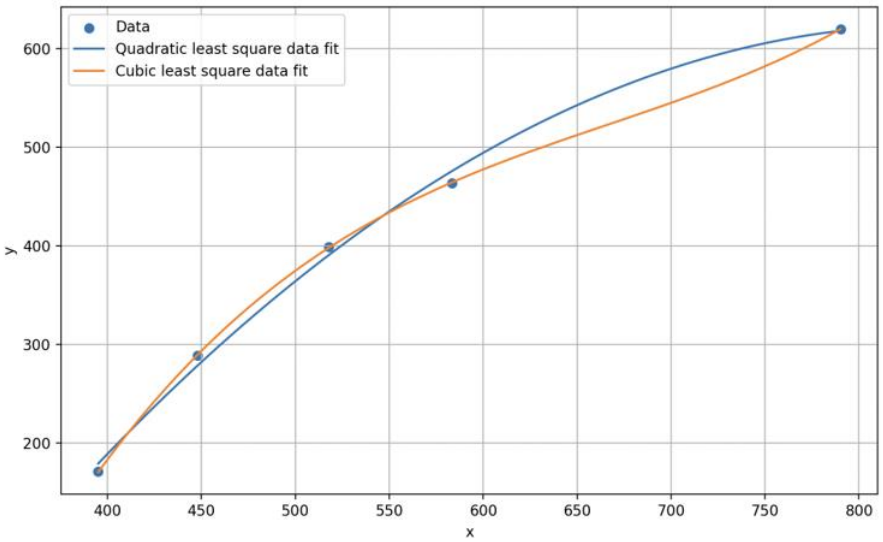


The spring constant (k) is approximately 2.52 N/cm
The displacement at which force is zero (intercept) is approximately -38.92 cm

```
import numpy as np
import matplotlib.pyplot as plt
# Data
y = np.array([0, 9, 18, 27])
x = np.array([15.5, 19.31, 22.1, 26.42])
# Calculating slope (k) and intercept (a) using least squares regression
n = len(y)
k = (n * np.sum(x * y) - np.sum(x) * np.sum(y)) / (n * np.sum(x**2) - (np.sum(x))**2)
a = (np.sum(y) - k * np.sum(x)) / n
# Generating points for the linear fit line
x_fit = np.linspace(min(x), max(x), 1000)
y_fit = a + k * x_fit
# Plotting
plt.figure(figsize=(10, 6))
plt.scatter(x, y, label="Data")
plt.plot(x_fit, y_fit, label="Linear Fit")
plt.xlabel('Displacement (cm)') plt.ylabel('Force (N)') plt.title('Hooke\'s Law')
plt.legend()
plt.grid(True)
plt.show()
# Printing results
```

```
print(f"The spring constant (k) is approximately {k:.2f} N/cm") print(f"The
displacement at which force is zero (intercept) is approximately {a:.2f} cm")
```

Problem 7



As we can see the Cubic least square data fit is more precise

```
import numpy as np
import matplotlib.pyplot as plt
def least_square_fit(x, y, x_fit, degree):
coefficients = np.polyfit(x, y, degree) return np.polyval(coefficients, x_fit)
x = np.array([395.1, 448.1, 517.7, 583.3, 790.2]) y = np.array([171.0, 289.0, 399.0,
464.0, 620.0])
x_fit = np.linspace(x.min(), x.max(), 1000)
plt.figure(figsize=(10, 6))
plt.scatter(x, y, label="Data")
plt.plot(x_fit, least_square_fit(x, y, x_fit, 2), label="Quadratic least square data
fit") plt.plot(x_fit, least_square_fit(x, y, x_fit, 3), label="Cubic least square
data fit")
plt.xlabel('x') plt.ylabel('y')
plt.grid(True)
plt.legend() plt.show()
```